

- Secrets
- ABAP
- Ansible
- Apex
- AzureResourceManager
- C
- C#
- C++
- CloudFormation
- COBOL
- CSS
- Dart
- Docker
- Flex
- Go
- HTML
- Java
- JavaScript
- JCL
- Kotlin
- Kubernetes
- Objective C
- PHP
- PL/I
- PL/SQL
- Python
- RPG
- Ruby
- Scala
- Swift
- Terraform
- Text
- TypeScript
- T-SQL
- VB.NET
- VB6
- XML



# Dart static code analysis

Unique rules to write Clean DART Code

All rules 115 Bug 15 Code Smell 100

Tags

Impact

Clean code attribute

Search by name...

|                                                                    |            |
|--------------------------------------------------------------------|------------|
| Spread collections should be preferred to chaining list insertions | Code Smell |
| Inline list literals should be preferred to chains of insertions   | Code Smell |
| Pubspec urls should be secure                                      | Code Smell |
| Referenced packages should be listed as dependencies               | Code Smell |
| For elements should be preferred to Map.fromIterable               | Code Smell |
| Fields should not be overridden                                    | Code Smell |
| Library prefixes shouldn't start with underscore                   | Code Smell |
| Private types shouldn't be exposed in public API                   | Code Smell |
| Library prefixes should comply with naming conventions             | Code Smell |
| Library annotations should be attached to library directive        | Code Smell |
| Library doc comments should be attached to library directive       | Code Smell |

## Spread collections should be preferred to chaining list insertions

Analyze your code

Intentionality - Efficient Maintainability

Code Smell Major

Chaining list(s) to a list literal via `List<E>.addAll` should be replaced by a literal with the `spread operator` (`...`).

- Why is this an issue?
- How can I fix it?
- More Info

Chaining list(s) to a list literal via `addAll`, e.g. `[ 42 ] .. addAll ( 11 ) .. addAll ( 11 ) ;`, is both less efficient and less verbose than using a list literal with the spread operator: `[ 42 , ... 11 , ... 11 ]`.

- `addAll` chains can be less efficient because each `addAll` call is treated as a separate method call, resulting in unnecessary invocations and allocations, when the goal is to build a single list with all the items provided as arguments
- `addAll` chains are generally more verbose and less clear because spreading a list can be done via dedicate syntax (`...`) which is closer to natural language

The rule also applies when the chained list is a `conditional expression`, because the spread operator can be applied conditionally:

```
[ 42 ] .. addAll ( i1 == i2 ? [ 43 , 44 ] : [] ); // Noncompliant
[ 42 , if ( i1 == i2 ) ... [ 43 , 44 ] ];      // Compliant
```

### Exceptions

The rule doesn't apply when the list chained is a literal, because `S7089` already covers this case.

```
[] .. addAll ( [ 42 ] ); // Not applicable
```

The rule doesn't cover `if and for collections` either, for the same reason (`S7089` covers it).

```
[ 42 ] .. addAll ( [ if ( i1 == 42 ) 43 , 44 ] ); // Not applicable
[ 42 ] .. addAll ( [ for ( final i in [ 1 , 2 , 3 ] ) i * i ] ); // Not applicable
```

The rule doesn't apply when the `addAll` is not called via cascade operator, but via standard dereference (`.`):

```
[] . addAll ( 11 ); // Not applicable
[] .. addAll ( 11 ); // Noncompliant
```

Available In:  
sonarcloud

